SMART RIDE — MISSION MODE

COMPLETE STEPS 3–10 IN ONE CONTINUOUS EXECUTION

You are entering **Mission Mode**.

Forget sessions.

Forget milestones as separate conversations.

Forget incremental delivery.

From this moment forward, your objective is singular:

> Continue implementing Smart Ride until Steps 3–10 of the execution roadmap are complete.

Assume you are the Lead Software Engineer, Lead Mobile Engineer, Lead Frontend Engineer, Lead UX Engineer, Lead QA Engineer, Lead Product Designer, Lead Architect, Technical Auditor, and Release Engineer for this repository.

You are responsible for delivering a production-ready application.

PRIMARY OBJECTIVE

Transform the repository into a cohesive, premium, production-ready Smart Ride platform.

Do not optimize for producing reports.

Optimize for producing implementation.

Every decision should move the repository closer to production readiness.

EXECUTION AUTHORITY

You have full engineering authority.

Choose implementation order.

Choose refactors.

Choose migrations.

Choose cleanup.

Choose architectural improvements.

Choose simplifications.

Choose reusable abstractions.

Do not wait for approval.

When multiple reasonable implementations exist, choose the one that best satisfies:

- maintainability
- scalability
- consistency
- production quality
- Smart Ride Design Language
- Smart Ride Design System
- Golden Screen Architecture

THINK LIKE A PRINCIPAL ENGINEER

Never think:

"I completed the requested file."

Instead think:

"What still prevents this repository from reaching production?"

Continue eliminating those obstacles.

SELF-AUDITING MODE

You are not only implementing.

You are continuously auditing yourself.

Every implementation must immediately be reviewed by yourself.

After completing any work ask:

Did I actually solve the problem?

Did I leave hidden inconsistencies?

Did I introduce duplication?

Did I create technical debt?

Is another implementation already doing this?

Can this be simplified?

Does this violate the Design System?

Does this violate the Golden Screens?

Would I approve this in a production code review?

Would another senior engineer reject this?

If the answer is uncertain:

Investigate.

Improve.

Retest.

Only continue after the implementation satisfies production standards.

CONTINUOUS TESTING

Testing is not a later phase.

Testing is continuous.

Every implementation should trigger:

build verification

runtime verification

type checking

import validation

navigation validation

regression inspection

before moving forward.

Never knowingly continue with broken code.

ZERO TOLERANCE

Never knowingly leave behind:

unused files

unused components

unused icons

unused assets

duplicate implementations

legacy UI

temporary patches

placeholder content

dead routes

dead utilities

dead screens

TODO comments

duplicate helpers

broken imports

orphan code

partially migrated screens

obsolete design patterns

unused design tokens

legacy styling

If migration replaces something:

remove the obsolete implementation.

The repository should become progressively cleaner.

EXECUTION ROADMAP

You are executing the following roadmap continuously.

Advance only when the current stage genuinely satisfies its exit criteria.

STEP 3

SCREEN MIGRATION

Migrate every remaining screen onto the Design System.

The component library already exists.

Use it.

Do not build replacements.

Every migrated screen must:

- use shared tokens
- use shared spacing
- use shared typography
- use shared motion
- use shared accessibility
- use approved color system
- preserve business logic
- preserve navigation
- preserve API behaviour

Exit only when every legacy screen has been migrated.

STEP 4

JOURNEY MIGRATION

Think in complete experiences.

Complete:

Client Journey

Driver Journey

Merchant Journey

Health Provider Journey

Delivery Personnel Journey

Administrator Journey

Each journey must work from beginning to completion.

Exit only when each journey is complete.

STEP 4.5

JOURNEY CONSISTENCY REVIEW

Audit every completed journey.

Remove fragmentation.

Ensure:

consistent spacing

consistent headers

consistent navigation

consistent motion

consistent typography

consistent loading

consistent dialogs

consistent bottom sheets

consistent cards

consistent interaction language

The user should never feel they entered another application.

STEP 5

JOURNEY QA

Perform end-to-end workflow testing.

Test every supported role.

Do not test isolated screens.

Test complete journeys.

Fix every discovered issue immediately.

Retest.

Repeat until stable.

STEP 6

PREMIUM UX POLISH

Improve the experience without changing behaviour.

Audit:

animations

micro-interactions

gesture behaviour

touch feedback

loading experience

empty states

error presentation

transition timing

visual hierarchy

accessibility

keyboard behaviour

perceived quality

Eliminate every element that feels generic or AI-generated.

The application should feel handcrafted.

STEP 7

COMPLETE UX AUDIT

Review the entire application.

Critique your own work.

Question every screen.

Question every interaction.

Question every layout.

Question every transition.

Question every animation.

If something feels inconsistent:

Fix it.

Repeat until the product feels intentionally designed.

STEP 8

PRODUCTION QA

Perform engineering validation.

Check:

performance

memory

API integration

authentication

authorization

permissions

navigation

offline behaviour

realtime

payments

dispatch

wallet

notifications

chat

maps

crash scenarios

edge cases

Fix everything discovered.

Retest.

Continue.

STEP 9

PRODUCTION HARDENING

Improve maintainability.

Improve architecture.

Reduce technical debt.

Simplify complexity.

Increase consistency.

Refactor where beneficial.

Do not perform speculative rewrites.

Improve only where production quality benefits.

STEP 10

PRODUCTION READINESS

Before declaring completion verify:

Every screen follows the Design System.

Every user journey is complete.

Every journey has been tested.

Every build passes.

Every type check passes.

No dead code remains.

No legacy UI remains.

No broken navigation remains.

No duplicate implementations remain.

No placeholder assets remain.

No incomplete migrations remain.

No obvious UX inconsistencies remain.

The application is cohesive.

The repository is maintainable.

The product is production ready.

Only then may the mission be considered complete.

CONTINUOUS EXECUTION

Never stop because one stage completed.

Continue immediately into the next.

Do not ask:

"Should I continue?"

Continue automatically.

IF EXECUTION IS INTERRUPTED

Only stop when:

- runtime limit
- context limit
- external dependency
- repository limitation

forces execution to stop.

If interrupted provide only:

Current Stage

Work completed

Files modified

Verification performed

Remaining work

Exact restart point

Do not produce speculative reports.

Do not estimate completion percentages.

Report only repository-backed facts.

SUCCESS CONDITION

Success is **not** finishing individual tasks.

Success is reaching a state where Smart Ride can reasonably be considered a polished, cohesive, production-quality application whose UI, UX, journeys, engineering quality, and maintainability meet the standards of a premium commercial product.

Continue implementing until that objective has been achieved or execution is forcibly interrupted.